

Universidad ORT Uruguay

Facultad de ingeniería Bernand

Wand-Polak

**Obligatorio 1 Estructuras de
datos y algoritmos II**

Juan Pedro Albanell

(311570)

Ignacio Miranda

(308962)

Profesor: Joaquin Vigna

Grupo M4B

23/10/2025

Diseño e implementación EJERCICIO 1

Estructuras

- NodoAVL (índice por ID): punteros izq/der, id, pts, nombre, altura.
- NodoAVLpts (índice por puntaje):
punteros izq/der, pts, cant, cantMayorPts, altura.
- RepresentacionAVL: raizID, raizPTS, cantReg, nomMejorJug, ptsMejorJug, idMejorJug.

Operaciones

- **ADD id nombre puntaje**
 1. Inserta en raizID si el id no existe (rotaciones AVL clásicas para balanceo).
 2. Si se insertó, incrementa cantReg, actualiza raizPTS (si hay mismo puntaje, aumenta cant) y mantiene los acumulados (cantMayorPts).
 3. Actualiza **TOP1**: si el nuevo puntaje es mayor, o si empata pero el id es menor, se reemplaza el líder.
- **FIND id**

Búsqueda binaria sobre raizID; devuelve "nombre puntaje" o "jugador_no_encontrado".

- **RANK p**

En raízPTS: si $a \rightarrow pts \geq p$, suma **todo el subárbol derecho** + cant del nodo +
recurre a izquierda; si $a \rightarrow pts < p$, recurre a derecha. El
acumulado cantMayorPts permite contar en $O(1)$ el tamaño de cada subárbol
durante el descenso.

- **TOP1**

Devuelve el líder mantenido en la representación. En empate por puntaje,
gana el de **menor id**.

- **COUNT**

Devuelve cantReg actualizado en cada inserción. **Justificación de
complejidad temporal**

Operación	Estructura	Complejidad	Justificación
ADD	AVL por ID y AVL por puntaje	$O(\log N)$	Inserción AVL balanceada por ID ($O(\log N)$); si se insertó, se actualiza el AVL por puntaje (clave = valor de puntaje) también en $O(\log N)$. Rotaciones y actualización de alturas/contadores son $O(1)$.
FIND	AVL por ID	$O(\log N)$	Búsqueda binaria en AVL balanceado por id.
RANK(p)	AVL por puntaje	$O(\log N)$	Descenso por el AVL de puntajes usando sumas de tamaño de subárbol (cantMayorPts) para acumular en $O(1)$ por nivel. Altura $O(\log N)$.
TOP1	Campos globales	$O(1)$	El líder se mantiene al momento del ADD (con desempate por menor id).
COUNT	Campo global	$O(1)$	cantReg se incrementa sólo cuando se agrega un id nuevo

Diseño e implementación EJERCICIO 2

Estructuras

- **Tablas hash cerradas con doble hashing:**
 - **tablaDomPath[]** (clave compuesta dom+path) con vector paralelo **tablaDomPathSeBorro[]** (marca de borrado). Cada celda guarda **NodoHashDomPath{dom, path, titulo, tiempo}**. El doble hashing usa $h1 = \text{hash3}(\dots)$ y $h2 = 1 + \text{hashSec}(\dots) \bmod (B-1)$.
 - **tablaDom[]** para indexar por **dominio**, con **tablaDomSeBorro[]** y **cantDom[]** (contador por bucket de dominio). Cada bucket apunta al **head de una lista enlazada** de **NodoHashDom{dom, path, titulo, tiempo, sig}** para ese dominio (sirve a LIST_DOMAIN).
- **Contadores globales:** **cantElem** (tamaño total del cache) y **buckets** (capacidad).

Operaciones

- **PUT(d, p, t, ti)**
 1. Localiza/inserta el **dominio** en **tablaDom** (doble hashing).
 2. Inserta un nodo **al frente** de la lista del dominio; si el path ya existía, **se elimina su ocurrencia previa**.
 3. Actualiza/crea entrada en **tablaDomPath** (si existe, sólo cambia titulo/tiempo; si no, inserta en celda libre (respetando eliminados)).
 4. Ajusta **cantDom[posDom]** y **cantElem** **solo si fue agregado**.

- **GET(d, p)**

Busca (d,p) en tablaDomPath; imprime 'titulo tiempo' o recurso_no_encontrado.

- **REMOVE(d, p)**

Elimina (d,p) de tablaDomPath marcando **eliminado** y quita el nodo correspondiente de la **lista del dominio**; actualiza contadores.

- **CONTAINS(d, p)**

Devuelve true/false consultando tablaDomPath.

- **COUNT_DOMAIN(d)**

Localiza el bucket de d y retorna cantDom[pos].

- **LIST_DOMAIN(d)**

Recorre la **lista del dominio** e imprime los path **más reciente → más antiguo**; si no hay, imprime **línea vacía**. (*Orden por recencia de operación, no por el valor tiempo.*)

- **CLEAR_DOMAIN(d)**

Borra la lista del dominio y, por cada nodo, elimina (d,p) de tablaDomPath (marca de borrado). Restablece contadores y marca tablaDomSeBorro[pos].

- **SIZE**

SIZE retorna cantElem

- **CLEAR**

CLEAR itera buckets de dominios y aplica CLEAR_DOMAIN.

Justificación de complejidad temporal

Operación	Estructura involucrada	Complejidad	Justificación breve
SIZE	Contador global cantElem	O(1) (peor caso)	Se retorna un contador mantenido en cada alta/baja.
GET(d,p)	Hash cerrado (clave dom+path) con dobles hashing	O(1) (prom.)	Búsqueda por sondeo; las “marca de borrado” preservan la secuencia.
CONTAINS(d,p)	Hash cerrado (clave dom+path)	O(1) (prom.)	Igual que GET pero booleana.
COUNT_DOMAIN(d)	Hash cerrado por dominio + cantDom[paths]	O(1) (prom.)	Localiza el bucket del dominio y lee su contador.
PUT(d,p,t,ti)	Hash (dom+path) + lista por dominio	O(k) (prom.)	Actualiza/crea en hash (O(1) prom.) y reubica/elimina en la lista del dominio (recorrido de hasta k paths).
REMOVE(d,p)	Hash (dom+path) + lista por dominio	O(k) (prom.)	Marca la marca de borrado en

			hash y elimina nodo de la lista del dominio.
LIST_DOMAIN(d)	Lista del dominio	O(k) (prom.)	Recorrido lineal de los paths del dominio en orden de recencia.
CLEAR_DOMAIN(d)	Hash (dom+path) + lista del dominio	O(k) (prom.)	Elimina todas las entradas del dominio y vacía su lista.
CLEAR	Todas las estructuras	O(M + R)	Recorre M buckets de dominios y borra R recursos totales.

- K = cantidad de recursos del dominio consultado.
- M = tamaño de la tabla por dominios (número de buckets).
- R = cantidad total de recursos en el cache.

Diseño e implementación EJERCICIO 3

Estructuras:

- **Tablas hash cerradas con doble hashing:**
 - **tablaDomPath[]** (clave compuesta dom+path), implementada como hash abierto con un array de tamaño B donde cada bucket es el head de una lista enlazada de NodoHashDomPath para los recursos. Cada NodoHashDomPath almacena dom, path, titulo, tiempo y punteros **sig** (siguiente en el bucket).
 - **tablaDom[]** para indexar por dominio, implementada como hash abierto: cada bucket apunta a una lista enlazada de dominios. Cada NodoHashDom almacena dom, path, titulo, tiempo y punteros **sig** (siguiente en el bucket).
 - **tablaCantDom[]** para indexar por dominio, implementada como hash abierto: cada bucket apunta a una lista enlazada de NodoCantDom. Cada NodoCantDom almacena dom, cant y punteros **sig** (siguiente en el bucket).
 - **Contadores globales:** cantElem (tamaño total del cache) y buckets (capacidad de tablaDomPath).

Operaciones

- **PUT(d, p, t, ti)**

Localiza o inserta el dominio en tablaDom (hash abierto). Inserta un nuevo nodo al frente de la lista del dominio; si el path ya existía, se elimina su ocurrencia previa antes de agregar el nuevo nodo. Actualiza o crea la entrada en tablaDomPath (si existe, solo actualiza título/tiempo; si no, inserta el nuevo nodo en la lista enlazada del bucket correspondiente). Ajusta el contador del dominio y cantElem solo si se agregó un recurso nuevo.

- **GET(d, p)**

Busca (d,p) en tablaDomPath recorriendo la lista del bucket; imprime 'título tiempo' o recurso_no_encontrado.

- **REMOVE(d, p)**

Elimina (d,p) de tablaDomPath quitando el nodo de su lista de colisión y removiéndolo de la lista del dominio; actualiza los contadores correspondientes. Si un dominio quedó sin recursos, se elimina su nodo de tablaDom.

- **CONTAINS(d, p)**

Devuelve true/false consultando tablaDomPath.

- **COUNT_DOMAIN(d)**

Localiza el bucket de d y retorna su contador.

- **LIST_DOMAIN(d)**

Recorre la lista de recursos del dominio d (enlazada por sigDom) e imprime los paths del más reciente al más antiguo; si el dominio no existe o no tiene recursos, imprime una línea vacía.

- **CLEAR_DOMAIN(d)**

Elimina todos los recursos del dominio d: recorre la lista del dominio y remueve cada nodo tanto de tablaDomPath (lista de colisión) como de la lista del dominio. Actualiza cantElem y elimina el nodo del dominio en tablaDom.

- **SIZE**

SIZE retorna cantElem

- **CLEAR**

Elimina todos los recursos del caché: recorre todos los dominios en tablaDom y aplica CLEAR_DOMAIN a cada uno.

Justificación de complejidad temporal

Operación	Estructura involucrada	Complejidad	Justificación breve
SIZE	Contador global cantElem	O(1) (peor caso)	Se retorna un contador mantenido en cada alta/baja.
GET(d,p)	Hash abierto (clave dom+path) con lista enlazada	O(1) (prom.)	Búsqueda en la lista de colisión del bucket correspondiente (promedio).
CONTAINS(d,p)	Hash abierto (clave dom+path)	O(1) (prom.)	Igual que GET pero booleana.
COUNT_DOMAIN(d)	Hash abierto por dominio	O(1) (prom.)	Busca el nodo del dominio en su lista y lee su contador.
PUT(d,p,t,ti)	Hash (dom+path) + lista por dominio	O(k) (prom.)	Inserción/actualización en hash (O(1) prom.) y reubicación/eliminación en la lista del dominio (hasta k nodos a recorrer).
REMOVE(d,p)	Hash (dom+path) + lista por dominio	O(k) (prom.)	Remueve el nodo tanto del hash

			(lista enlazada) como de la lista del dominio.
LIST_DOMAIN(d)	Lista del dominio	O(k) (prom.)	Recorrido lineal de los k paths del dominio en orden de recencia.
CLEAR_DOMAIN(d)	Hash (dom+path) + lista del dominio	O(k) (prom.)	Elimina todas las entradas del dominio y vacía su lista.
CLEAR	Todas las estructuras	O(M + R)	Recorre M bucke ts de dominios y borra R recursos totales.

- k = cantidad de recursos del dominio consultado.
- M = tamaño de la tabla por dominios (número de buckets).
- R = cantidad total de recursos en el caché.

Diseño e implementación EJERCICIO 4

Estructuras:

- **Listas enlazadas ordenadas:** se tienen k listas enlazadas de números enteros ordenados. Cada lista se compone de nodos con campo valor y puntero al siguiente nodo.
- **MinHeap:** estructura binaria mínima implementada sobre un arreglo, capaz de almacenar hasta k elementos y de extraer el menor en $O(\log k)$. Se utiliza para obtener el siguiente valor más pequeño entre las listas.
- **Lista resultado:** lista enlazada donde se acumulan en orden creciente todos los elementos fusionados de las k listas de entrada. Inicialmente vacía.

Operaciones

- **agregarFinalLista(H, l, e)**
Agrega un nodo con valor e al final de la lista l . Si la lista aún no existe, la crea e inicializa.
- **cabezal(H, l)**
Devuelve el primer nodo (cabeza) de la lista l y actualiza el puntero raíz al siguiente nodo. Si la lista queda vacía, también actualiza final.
- **agregarHeap($H, nodo$)**
Inserta un nuevo nodo en el montículo y restablece la propiedad de heap utilizando la operación flotar. Si el heap está lleno o el nodo es NULL, no realiza ninguna acción.

- **flotar(H, pos)**

Restaura la propiedad de min-heap ascendiendo el nodo en la posición pos mientras sea menor que su padre.

- **hundir(H, pos)**

Restaura la propiedad de min-heap descendiendo el nodo en la posición pos al lugar correcto comparándolo con sus hijos.

- **eliminar(H)**

Extrae el nodo con el valor mínimo (en la raíz del heap), lo elimina y reordena el heap mediante hundir. Luego inserta en el heap el siguiente nodo de la misma lista de la cual provino el nodo eliminado (si existe), garantizando que siempre haya al menos un candidato de cada lista mientras haya elementos restantes.

- **esVacio(H)**

Retorna true si el heap está vacío (es decir, si proxPosLibre == 1), o false en caso contrario.

Algoritmo de fusión

- 1- Leer el número de listas K, para cada lista, su tamaño N y sus elementos en orden creciente. Insertar los elementos en la estructura arrayListas mediante agregarFinalLista.
- 2- Insertar el primer nodo de cada lista en el heap utilizando agregarHeap.
- 3- Mientras el heap no esté vacío:
 - Extraer el nodo con menor valor utilizando eliminar.
 - imprimir su valor.
 - Insertar en el heap el siguiente nodo de la lista correspondiente (si existe).

4- El proceso termina cuando todos los nodos han sido procesados y el heap queda vacío.

Justificación de complejidad temporal

Operación	Estructura involucrada	Complejidad	Justificación breve
crearHeap(K)	MinHeap	O(K)	Reserva memoria e inicializa estructuras aux.
agregarHeap	MinHeap	O(1) (cp) / O(log k) pc	Insertar en un heap tiene orden 1 cp y orden log k pc (por la cantidad de niveles del arbol binario)
eliminar	MinHeap	O(log k) pc	eliminar en un heap tiene orden (log k) pc (por la cantidad de niveles del árbol binario)
flotar/hundir	MinHeap	O(log k) pc	Restaura la propiedad del heap recorriendo la altura del árbol.
fusión completa	MinHeap + listas enlazadas	O(Nlog k)	Se realizan <i>N</i> inserciones y extracciones en el heap, cada

			una con costo $O(\log K)$.
--	--	--	--------------------------------

- K = número de listas ordenadas.
- N = número total de elementos en todas las listas combinadas.

Diseño e implementación EJERCICIO 5

Estructuras

Para resolver el problema del Árbol o Bosque de Cubrimiento Mínimo se implementó el algoritmo de Kruskal utilizando las siguientes estructuras:

- MinHeap de aristas: estructura que permite obtener en todo momento la arista de menor peso. Cada nodo contiene un puntero a una estructura con los campos $v1$, $v2$ y peso. El heap se utiliza para ordenar las aristas implícitamente y poder extraerlas en $O(\log E)$.
- MFset (Disjoint Set o Union-Find optimizado): estructura utilizada para mantener las componentes conexas del grafo durante la ejecución de Kruskal.

- padre[]: indica el padre de cada vértice.
- altura[]: guarda la altura aproximada de cada árbol.
- tam: cantidad total de vértices.

Se implementan dos optimizaciones fundamentales:

- Compresión de caminos en la operación buscar, que aplanar los árboles de representación.
- Unión por altura en la operación unir, que enlaza el árbol más bajo bajo el más alto.

Operaciones

crear(n)

Inicializa la estructura de conjuntos disjuntos (MFset) creando arreglos padre y altura de tamaño $n+1$. Cada vértice comienza como su propio conjunto, con altura inicial 1.

buscar(mf, x)

Devuelve el representante del conjunto al que pertenece x . Si el vértice no es su propio padre, se aplica recursión y se actualiza padre[x] para apuntar directamente al representante (compresión de caminos).

`unir(mf, a, b)`

Une los conjuntos de a y b si pertenecen a diferentes componentes. Se usa unión por altura: el árbol de menor altura se conecta al de mayor altura. En caso de empate, se incrementa la altura del nuevo padre.

Kruskal con heap de aristas

1. Se leen los valores V (vértices) y E (aristas).
2. Se crea la estructura MFset y un MinHeap.
3. Se cargan todas las aristas en el heap, representadas por estructuras con sus respectivos vértices y pesos.
4. Se inicializa una variable para almacenar el peso total.
5. Mientras el heap no esté vacío, se toma la arista mínima (raíz del heap):
 - Si $\text{unir}(v1, v2)$ retorna verdadero (no forma ciclo), se suma su peso al total.
 - Se elimina la arista mínima del heap.
6. Finalmente, se imprime el peso total, correspondiente al árbol de cubrimiento mínimo (o bosque, si el grafo no es conexo).

Operación	Estructura	Complejidad	Justificación
crear(V)	Union-Find	$O(V)$	Inicialización de los arreglos padre y altura.
buscar(x)	Union-Find		Compresión de caminos: casi constante en la práctica.
unir(a, b)	Union-Find		Unión por altura mantiene árboles casi planos.
agregarHeap(E)	MinHeap	$O(E \log E)$	Inserción de las aristas en el heap.
extraer mínimo	MinHeap	$O(\log E)$	Reordenamiento del heap tras la extracción.
Kruskal total	MinHeap + Union-Find	$O(E \log E)$	E inserciones, E extracciones y E operaciones de unión/búsqueda. El ordenamiento domina.

Justificación de complejidad temporal Donde:

- V es el número de vértices del grafo.
- E es el número de aristas del grafo.

Tabla con resultados obtenidos en cada ejercicio:

Problema	Resultado
1	Completo
2	Completo
3	Completo
4	Completo
5	Completo
6	
7	
8	
9	
10	

AUTORIA

Recorrido AVL – Basado en diapositivas de clase.

Funciones de Hash (Hash3) – Exacta como diapositivas de clase.

Funciones de Hash (HashSec) – Basada en diapositivas de clase.

Funciones heap – Basadas en diapositivas de clase, no copiadas.

Funciones MFSet – Basadas en diapositivas de clase, no copiadas.